

4. SotilityContentGovernance.sol (continued)

Plain Text

```
struct ModeratorStats {
    uint256 totalVotes;
    uint256 alignedVotes; // Votes that aligned with final decision
    uint256 stakeAmount;
    uint256 rewardsClaimed;
}

// Content registry
mapping(uint256 => Content) public contentRegistry;
uint256 public contentCount;

// Moderator statistics
mapping(address => ModeratorStats) public moderatorStats;
mapping(uint256 => mapping(address => ModeratorAction)) public
moderatorVotes;

// Economic parameters
uint256 public requiredStake; // Stake required to be a
moderator
uint256 public flagThreshold; // Number of flags to trigger
review
uint256 public moderatorRewardPool; // Total rewards available for
distribution
uint256 public rewardPerAlignedVote; // Reward per vote that aligns
with consensus
uint256 public slashingPercentage; // Percentage of stake slashed
for misalignment

// Events
event ContentRegistered(uint256 indexed contentId, address indexed
author, string contentHash);
event ContentFlagged(uint256 indexed contentId, address indexed
moderator);
event ContentStatusUpdated(uint256 indexed contentId, ContentStatus
newStatus);
event ModeratorVoted(uint256 indexed contentId, address indexed
moderator, ModeratorAction action);
event ModeratorRewarded(address indexed moderator, uint256 amount);
event ModeratorSlashed(address indexed moderator, uint256 amount);

constructor(
    uint256 _requiredStake,
    uint256 _flagThreshold,
    uint256 _rewardPerAlignedVote,
```

```

        uint256 _slashingPercentage
    ) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);

        requiredStake = _requiredStake;
        flagThreshold = _flagThreshold;
        rewardPerAlignedVote = _rewardPerAlignedVote;
        slashingPercentage = _slashingPercentage;
    }

    /**
     * @dev Register as a moderator by staking tokens
     */
    function registerAsModerator() external payable {
        require(msg.value >= requiredStake, "Insufficient stake");
        require(!hasRole(MODERATOR_ROLE, msg.sender), "Already a moderator");

        _grantRole(MODERATOR_ROLE, msg.sender);

        ModeratorStats storage stats = moderatorStats[msg.sender];
        stats.stakeAmount = msg.value;
    }

    /**
     * @dev Unregister as a moderator and withdraw stake
     */
    function unregisterAsModerator() external {
        require(hasRole(MODERATOR_ROLE, msg.sender), "Not a moderator");

        ModeratorStats storage stats = moderatorStats[msg.sender];
        uint256 stakeToReturn = stats.stakeAmount;

        _revokeRole(MODERATOR_ROLE, msg.sender);
        stats.stakeAmount = 0;

        payable(msg.sender).transfer(stakeToReturn);
    }

    /**
     * @dev Register new content
     * @param contentHash IPFS hash of the content
     */
    function registerContent(string calldata contentHash) external returns
(uint256) {
        contentCount++;

        contentRegistry[contentCount] = Content({
            id: contentCount,

```

```

        author: msg.sender,
        contentHash: contentHash,
        status: ContentStatus.Active,
        flagCount: 0,
        approvalCount: 0,
        lastDecisionTime: block.timestamp,
        moderatorsVoted: new address[](0)
    });

    emit ContentRegistered(contentCount, msg.sender, contentHash);
    return contentCount;
}

/**
 * @dev Cast a moderation vote on content
 * @param contentId ID of the content
 * @param action Moderation action to take
 */
function moderateContent(uint256 contentId, ModeratorAction action)
external onlyRole(MODERATOR_ROLE) {
    require(contentId <= contentCount, "Content does not exist");
    Content storage content = contentRegistry[contentId];

    // Check if moderator has already voted
    bool alreadyVoted = false;
    for (uint i = 0; i < content.moderatorsVoted.length; i++) {
        if (content.moderatorsVoted[i] == msg.sender) {
            alreadyVoted = true;
            break;
        }
    }
    require(!alreadyVoted, "Already voted on this content");

    // Record vote
    moderatorVotes[contentId][msg.sender] = action;
    content.moderatorsVoted.push(msg.sender);
    moderatorStats[msg.sender].totalVotes++;

    // Update content status based on action
    if (action == ModeratorAction.Flag) {
        content.flagCount++;
        if (content.flagCount >= flagThreshold && content.status ==
ContentStatus.Active) {
            content.status = ContentStatus.UnderReview;
            emit ContentStatusUpdated(contentId,
ContentStatus.UnderReview);
        }
    } else if (action == ModeratorAction.Approve) {

```

```

        content.approvalCount++;
    } else if (action == ModeratorAction.Remove &&
hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) {
        // Only admins can directly remove content
        content.status = ContentStatus.Removed;
        _processModerationOutcome(contentId, ContentStatus.Removed);
        emit ContentStatusUpdated(contentId, ContentStatus.Removed);
    } else if (action == ModeratorAction.Restore &&
hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) {
        // Only admins can directly restore content
        content.status = ContentStatus.Restored;
        _processModerationOutcome(contentId, ContentStatus.Restored);
        emit ContentStatusUpdated(contentId, ContentStatus.Restored);
    }

    emit ModeratorVoted(contentId, msg.sender, action);

    // Check for moderation consensus
    _checkModerationConsensus(contentId);
}

/**
 * @dev Check if consensus has been reached on content moderation
 * @param contentId ID of the content
 */
function _checkModerationConsensus(uint256 contentId) internal {
    Content storage content = contentRegistry[contentId];

    // Only process if under review
    if (content.status != ContentStatus.UnderReview) {
        return;
    }

    uint256 totalVotes = content.moderatorsVoted.length;
    if (totalVotes < 3) {
        return; // Need at least 3 votes for consensus
    }

    // Determine outcome based on votes
    if (content.flagCount > totalVotes / 2) {
        content.status = ContentStatus.Removed;
        _processModerationOutcome(contentId, ContentStatus.Removed);
        emit ContentStatusUpdated(contentId, ContentStatus.Removed);
    } else if (content.approvalCount > totalVotes / 2) {
        content.status = ContentStatus.Active;
        _processModerationOutcome(contentId, ContentStatus.Active);
        emit ContentStatusUpdated(contentId, ContentStatus.Active);
    }
}

```

```

        content.lastDecisionTime = block.timestamp;
    }

    /**
     * @dev Process rewards and slashing based on moderation outcome
     * @param contentId ID of the content
     * @param finalStatus Final status of the content
     */
    function _processModerationOutcome(uint256 contentId, ContentStatus
finalStatus) internal {
        Content storage content = contentRegistry[contentId];

        for (uint i = 0; i < content.moderatorsVoted.length; i++) {
            address moderator = content.moderatorsVoted[i];
            ModeratorAction vote = moderatorVotes[contentId][moderator];

            bool aligned = (
                (finalStatus == ContentStatus.Removed && vote ==
ModeratorAction.Flag) ||
                (finalStatus == ContentStatus.Active && vote ==
ModeratorAction.Approve) ||
                (finalStatus == ContentStatus.Restored && vote ==
ModeratorAction.Restore)
            );

            if (aligned) {
                // Reward aligned votes
                moderatorStats[moderator].alignedVotes++;
                _rewardModerator(moderator, rewardPerAlignedVote);
            } else {
                // Slash for misaligned votes
                _slashModerator(moderator);
            }
        }
    }

    /**
     * @dev Reward a moderator for aligned votes
     * @param moderator Address of the moderator
     * @param amount Amount to reward
     */
    function _rewardModerator(address moderator, uint256 amount) internal {
        require(moderatorRewardPool >= amount, "Insufficient reward pool");

        moderatorRewardPool -= amount;
        moderatorStats[moderator].rewardsClaimed += amount;
    }

```

```

        emit ModeratorRewarded(moderator, amount);
    }

    /**
     * @dev Slash a moderator for misaligned votes
     * @param moderator Address of the moderator
     */
    function _slashModerator(address moderator) internal {
        ModeratorStats storage stats = moderatorStats[moderator];
        uint256 slashAmount = (stats.stakeAmount * slashingPercentage) /
10000;

        if (slashAmount > 0) {
            stats.stakeAmount -= slashAmount;
            moderatorRewardPool += slashAmount; // Add slashed amount to
reward pool

            emit ModeratorSlashed(moderator, slashAmount);
        }
    }

    /**
     * @dev Claim accumulated rewards
     */
    function claimRewards() external {
        require(hasRole(MODERATOR_ROLE, msg.sender), "Not a moderator");

        ModeratorStats storage stats = moderatorStats[msg.sender];
        uint256 amount = stats.rewardsClaimed;
        require(amount > 0, "No rewards to claim");

        stats.rewardsClaimed = 0;

        payable(msg.sender).transfer(amount);
    }

    /**
     * @dev Add funds to the moderator reward pool
     */
    function addToRewardPool() external payable {
        moderatorRewardPool += msg.value;
    }
}

```

Financial Infrastructure Contracts

1. SotilityExchange.sol

Plain Text

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";

/**
 * @title SotilityExchange
 * @dev Tokenized listing and SST issuance for revenue-verified businesses.
 */
contract SotilityExchange is AccessControl {
    bytes32 public constant BUSINESS_ROLE = keccak256("BUSINESS_ROLE");
    bytes32 public constant VERIFIER_ROLE = keccak256("VERIFIER_ROLE");

    struct Business {
        string name;
        string description;
        string ipfsMetadata;
        address wallet;
        bool active;
    }

    struct RevenueProof {
        string ipfsHash;
        uint256 amount;
        uint256 timestamp;
    }

    mapping(address => Business) public businesses;
    mapping(address => RevenueProof[]) public revenueProofs;

    IERC20 public stableToken; // e.g. SST
    address public treasury;

    event BusinessRegistered(address indexed wallet, string name);
    event RevenueSubmitted(address indexed business, uint256 amount, string ipfsHash);
    event SSTIssued(address indexed to, uint256 amount);

    constructor(address _stableToken, address _treasury) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        stableToken = IERC20(_stableToken);
        treasury = _treasury;
    }
}
```

```

function registerBusiness(
    address wallet,
    string calldata name,
    string calldata description,
    string calldata ipfsMetadata
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(!businesses[wallet].active, "Already registered");

    businesses[wallet] = Business({
        name: name,
        description: description,
        ipfsMetadata: ipfsMetadata,
        wallet: wallet,
        active: true
    });

    _grantRole(BUSINESS_ROLE, wallet);

    emit BusinessRegistered(wallet, name);
}

function submitRevenueProof(address business, uint256 amount, string
calldata ipfsHash)
    external onlyRole(VERIFIER_ROLE)
{
    require(businesses[business].active, "Business not registered");

    revenueProofs[business].push(RevenueProof({
        ipfsHash: ipfsHash,
        amount: amount,
        timestamp: block.timestamp
    }));

    ERC20Burnable(address(stableToken)).mint(business, amount); //
    Assume minting rights exist

    emit RevenueSubmitted(business, amount, ipfsHash);
    emit SSTIssued(business, amount);
}

function getRevenueHistory(address business) external view returns
(RevenueProof[] memory) {
    return revenueProofs[business];
}

function deactivateBusiness(address business) external
onlyRole(DEFAULT_ADMIN_ROLE) {

```

```

        businesses[business].active = false;
    }

    function updateTreasury(address newTreasury) external
    onlyRole(DEFAULT_ADMIN_ROLE) {
        treasury = newTreasury;
    }
}

```

2. SotilityTreasuryRouter.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title SotilityTreasuryRouter
 * @dev Routes protocol revenue to SOT dividends, SST mint vaults, and SUG
 campaigns.
 */
contract SotilityTreasuryRouter is AccessControl {
    bytes32 public constant ROUTER_ROLE = keccak256("ROUTER_ROLE");

    address public dividendPool;
    address public stableVault;
    address public campaignPool;

    uint256 public splitToDividends = 4000; // 40%
    uint256 public splitToStable = 4000; // 40%
    uint256 public splitToCampaigns = 2000; // 20%

    event RevenueReceived(address indexed sender, uint256 amount);
    event RevenueSplit(uint256 toDividends, uint256 toVault, uint256
toCampaigns);
    event DestinationsUpdated(address dividendPool, address stableVault,
address campaignPool);
    event SplitsUpdated(uint256 toDividends, uint256 toVault, uint256
toCampaigns);

    constructor(address _dividendPool, address _stableVault, address
_campaignPool) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        dividendPool = _dividendPool;
        stableVault = _stableVault;
    }
}

```

```

        campaignPool = _campaignPool;
    }

    receive() external payable {
        emit RevenueReceived(msg.sender, msg.value);
        _split(msg.value);
    }

    function _split(uint256 amount) internal {
        uint256 toDividends = (amount * splitToDividends) / 10000;
        uint256 toVault = (amount * splitToStable) / 10000;
        uint256 toCampaigns = amount - toDividends - toVault;

        payable(dividendPool).transfer(toDividends);
        payable(stableVault).transfer(toVault);
        payable(campaignPool).transfer(toCampaigns);

        emit RevenueSplit(toDividends, toVault, toCampaigns);
    }

    function updateDestinations(address _dividend, address _vault, address
_campaign)
        external onlyRole(DEFAULT_ADMIN_ROLE)
    {
        dividendPool = _dividend;
        stableVault = _vault;
        campaignPool = _campaign;

        emit DestinationsUpdated(_dividend, _vault, _campaign);
    }

    function updateSplits(uint256 toDiv, uint256 toVault, uint256 toCamp)
        external onlyRole(DEFAULT_ADMIN_ROLE)
    {
        require(toDiv + toVault + toCamp == 10000, "Invalid split sum");

        splitToDividends = toDiv;
        splitToStable = toVault;
        splitToCampaigns = toCamp;

        emit SplitsUpdated(toDiv, toVault, toCamp);
    }
}

```

3. SotilityYieldEngine.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

/**
 * @title SotilityYieldEngine
 * @notice AI-managed, cross-chain, laddered-leverage staking vault for
stablecoins.
 */
contract SotilityYieldEngine is Ownable {
    struct Strategy {
        string name;
        address targetProtocol;
        uint256 allocationBps; // Allocation in basis points (1% = 100)
        uint256 leverageRatio; // Multiplier, e.g., 2 = 2x
        uint256 lockTime;      // Lock duration in seconds
    }

    IERC20 public stableToken;
    address public rewardDistributor;
    Strategy[] public strategies;

    mapping(address => uint256) public deposits;
    mapping(address => uint256) public entryTimestamps;

    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    event StrategyAdded(string name);

    constructor(address _stableToken, address _rewardDistributor) {
        stableToken = IERC20(_stableToken);
        rewardDistributor = _rewardDistributor;
    }

    function deposit(uint256 amount) external {
        require(amount > 0, "Amount must be greater than 0");

        stableToken.transferFrom(msg.sender, address(this), amount);
        deposits[msg.sender] += amount;
        entryTimestamps[msg.sender] = block.timestamp;

        emit Deposited(msg.sender, amount);
    }

    function withdraw(uint256 amount) external {
        require(deposits[msg.sender] >= amount, "Insufficient balance");
    }
}

```

```

        deposits[msg.sender] -= amount;
        stableToken.transfer(msg.sender, amount);

        emit Withdrawn(msg.sender, amount);
    }

    function addStrategy(
        string calldata name,
        address targetProtocol,
        uint256 allocationBps,
        uint256 leverageRatio,
        uint256 lockTime
    ) external onlyOwner {
        strategies.push(strategy({
            name: name,
            targetProtocol: targetProtocol,
            allocationBps: allocationBps,
            leverageRatio: leverageRatio,
            lockTime: lockTime
        }));

        emit StrategyAdded(name);
    }

    function rebalanceYieldStrategies() external onlyOwner {
        // Placeholder for AI strategy engine:
        // Fetch APYs, risk scores, reallocate across strategies
        // Could use Chainlink Functions, Gelato, or custom oracle
    }

    function getUserStakeDetails(address user) external view returns
    (uint256 staked, uint256 duration) {
        return (deposits[user], block.timestamp - entryTimestamps[user]);
    }

    function getStrategies() external view returns (Strategy[] memory) {
        return strategies;
    }

    function setRewardDistributor(address _distributor) external onlyOwner {
        rewardDistributor = _distributor;
    }
}

```

4. SotilityVaultFactory.sol

Plain Text

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import "../SotilityYieldEngine.sol";

/**
 * @title SotilityVaultFactory
 * @dev Deploys isolated yield engines for each verified project or business.
 */
contract SotilityVaultFactory is AccessControl {
    bytes32 public constant DEPLOYER_ROLE = keccak256("DEPLOYER_ROLE");

    struct VaultRecord {
        address creator;
        address vaultAddress;
        address token;
        address rewardDistributor;
        string ipfsMetadata;
        uint256 timestamp;
    }

    VaultRecord[] public deployedVaults;

    event VaultDeployed(address indexed vault, address token, address
distributor, string ipfsMetadata);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(DEPLOYER_ROLE, msg.sender);
    }

    function deployVault(
        address stableToken,
        address rewardDistributor,
        string calldata ipfsMetadata
    ) external onlyRole(DEPLOYER_ROLE) returns (address) {
        SotilityYieldEngine vault = new SotilityYieldEngine(stableToken,
rewardDistributor);

        deployedVaults.push(VaultRecord({
            creator: msg.sender,
            vaultAddress: address(vault),
            token: stableToken,
            rewardDistributor: rewardDistributor,
            ipfsMetadata: ipfsMetadata,
            timestamp: block.timestamp
        }));
    }
}
```

```

        emit VaultDeployed(address(vault), stableToken, rewardDistributor,
ipfsMetadata);
        return address(vault);
    }

    function getAllVaults() external view returns (VaultRecord[] memory) {
        return deployedVaults;
    }
}

```

5. SotilityBridgeAdapter.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title SotilityBridgeAdapter
 * @dev Bridges SST/SUG/SOT tokens across multiple chains.
 * Future-compatible with AI + fraud detection.
 */
contract SotilityBridgeAdapter is AccessControl {
    bytes32 public constant BRIDGE_OPERATOR_ROLE =
keccak256("BRIDGE_OPERATOR_ROLE");

    struct Destination {
        string chainName;
        address destinationBridge;
        uint64 chainId;
        bool enabled;
    }

    IERC20 public token;
    mapping(uint64 => Destination) public destinations;

    event DestinationAdded(uint64 chainId, string chainName, address bridge);
    event TokenBridged(address indexed sender, uint64 destinationChainId,
uint256 amount, string reason);
    event BridgeSuspicious(address indexed sender, string reason);

    constructor(address _token) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        token = IERC20(_token);
    }
}

```

```

}

function addDestination(
    uint64 chainId,
    string calldata name,
    address bridgeAddress
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    destinations[chainId] = Destination({
        chainName: name,
        destinationBridge: bridgeAddress,
        chainId: chainId,
        enabled: true
    });

    emit DestinationAdded(chainId, name, bridgeAddress);
}

function bridgeToken(uint64 chainId, uint256 amount, string calldata
reason) external {
    Destination memory dest = destinations[chainId];
    require(dest.enabled, "Destination disabled");

    require(token.transferFrom(msg.sender, address(this), amount),
"Transfer failed");

    // Placeholder logic: event emits for LayerZero or relayer bot to
pick up
    emit TokenBridged(msg.sender, chainId, amount, reason);
}

// Extension hook: flag suspicious activity
function flagSuspicious(address sender, string calldata reason) external
onlyRole(BRIDGE_OPERATOR_ROLE) {
    emit BridgeSuspicious(sender, reason);
}

function getDestination(uint64 chainId) external view returns
(Destination memory) {
    return destinations[chainId];
}
}

```

6. SotilityAdaptiveTokenomics.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityAdaptiveTokenomics
 * @dev Implements adaptive tokenomics based on ecosystem health metrics
 */
contract SotilityAdaptiveTokenomics is AccessControl {
    bytes32 public constant ORACLE_ROLE = keccak256("ORACLE_ROLE");
    bytes32 public constant PARAMETER_SETTER_ROLE =
keccak256("PARAMETER_SETTER_ROLE");

    // Ecosystem health metrics
    struct EcosystemState {
        uint256 userGrowthRate; // basis points (10000 = 100%)
        uint256 retentionRate; // basis points
        uint256 avgTransactionValue; // in smallest unit
        uint256 contentCreationRate; // per day
        uint256 totalValueLocked; // in USD (6 decimals)
        uint256 timestamp;
    }

    // Tokenomic parameters that can adapt
    struct TokenomicParameters {
        uint256 sugInflationRate; // basis points per year
        uint256 sugContentReward; // base reward in SUG tokens (18
decimals)
        uint256 sotDividendRatio; // percentage of revenue for dividends
(basis points)
        uint256 sstMintingThreshold; // minimum verified revenue for SST
minting (in USD, 6 decimals)
        uint256 veTokenBoostMax; // maximum boost for veToken holders
(basis points)
        uint256 timestamp;
    }

    // Current and historical states
    EcosystemState public currentState;
    TokenomicParameters public currentParameters;

    // History tracking
    EcosystemState[] public stateHistory;
    TokenomicParameters[] public parameterHistory;

    // Event emissions
    event EcosystemStateUpdated(uint256 indexed timestamp, uint256

```

```

stateIndex);
    event TokenomicParametersUpdated(uint256 indexed timestamp, uint256
paramIndex);

    constructor(
        uint256 initialInflationRate,
        uint256 initialContentReward,
        uint256 initialDividendRatio,
        uint256 initialMintingThreshold,
        uint256 initialVeTokenBoost
    ) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);

        // Initialize with default parameters
        currentParameters = TokenomicParameters({
            sugInflationRate: initialInflationRate,
            sugContentReward: initialContentReward,
            sotDividendRatio: initialDividendRatio,
            sstMintingThreshold: initialMintingThreshold,
            veTokenBoostMax: initialVeTokenBoost,
            timestamp: block.timestamp
        });

        parameterHistory.push(currentParameters);
    }

    /**
     * @dev Update the ecosystem state metrics
     * @param userGrowth New user growth rate
     * @param retention User retention rate
     * @param avgTxValue Average transaction value
     * @param contentRate Content creation rate
     * @param tvl Total value locked
     */
    function updateEcosystemState(
        uint256 userGrowth,
        uint256 retention,
        uint256 avgTxValue,
        uint256 contentRate,
        uint256 tvl
    ) external onlyRole(ORACLE_ROLE) {
        currentState = EcosystemState({
            userGrowthRate: userGrowth,
            retentionRate: retention,
            avgTransactionValue: avgTxValue,
            contentCreationRate: contentRate,
            totalValueLocked: tvl,
            timestamp: block.timestamp

```

```

    });

    stateHistory.push(currentState);

    emit EcosystemStateUpdated(block.timestamp, stateHistory.length - 1);

    // Automatically adjust tokenomics based on new state
    _adaptTokenomics();
}

/**
 * @dev Manually update tokenomic parameters (governance override)
 */
function setTokenomicParameters(
    uint256 inflationRate,
    uint256 contentReward,
    uint256 dividendRatio,
    uint256 mintingThreshold,
    uint256 veTokenBoost
) external onlyRole(PARAMETER_SETTER_ROLE) {
    currentParameters = TokenomicParameters({
        sugInflationRate: inflationRate,
        sugContentReward: contentReward,
        sotDividendRatio: dividendRatio,
        sstMintingThreshold: mintingThreshold,
        veTokenBoostMax: veTokenBoost,
        timestamp: block.timestamp
    });

    parameterHistory.push(currentParameters);

    emit TokenomicParametersUpdated(block.timestamp,
parameterHistory.length - 1);
}

/**
 * @dev Internal function to adapt tokenomics based on ecosystem state
 */
function _adaptTokenomics() internal {
    // Example adaptation logic:

    // 1. Adjust SUG inflation based on user growth and content creation
    uint256 newInflationRate;
    if (currentState.userGrowthRate > 500) { // >5% growth
        // Increase inflation to reward growth
        newInflationRate = currentParameters.sugInflationRate + 100; //
+1%
    } else {

```

```
        // Decrease inflation to reduce token supply growth
        newInflationRate = currentParameters.sugInflationRate > 200 ?
            currentParameters.sugInflationRate - 100 :
100; // min 1%
    }

    // 2. Adjust content rewards based on content creation rate
    uint256 newContentReward;
    if
```